

Numerical Methods Applied to Electrostatic Field Computation

ELECTIVE PROJECT AND COMPUTATION (High Voltage Technique)



Name: Uttam Kanti Dutta

Roll: 002110801216

Year: 2024-25

BEE UG IV B3

**Department of Electrical Engineering
Jadavpur University**

CO-Workers Name:

Riddhi Sinha	002110801242
Arka Saha	002110801222
Sandipan Naskar	002110801230
Soumyadeep Majee	302210801006
Nilava Biswas	002110801237

Introduction

Electrostatics, a key branch of physics, explores the forces and fields generated by stationary electric charges. It has wide-ranging applications, including the design of capacitors and insulators, as well as advancements in high-voltage engineering and electronics. A fundamental challenge in this field is determining the electric potential distribution between capacitor plates. When analytical solutions are not feasible due to complex geometries or boundary conditions, numerical approaches like the Finite Difference Method (FDM) become invaluable for solving such problems.

Finite Difference Method (FDM)

The Finite Difference Method (FDM) is a numerical technique for solving differential equations by approximating derivatives with finite differences. This method transforms partial differential equations (PDEs) into a system of algebraic equations that can be solved computationally. FDM is widely used in physics and engineering due to its simplicity and effectiveness in handling boundary value problems.

Capacitor Basics

A capacitor is an electrical component that stores energy in an electric field. Common applications of capacitors include energy storage in electronic devices, power conditioning in electrical systems, signal filtering in communication circuits, and as critical components in sensors and actuators. It consists of two conductive plates separated by a dielectric material. When a potential difference is applied across the plates, an electric field is established, and charges accumulate on the plates. The potential distribution in the field determines the capacitor's performance and is critical for designing efficient and reliable capacitive systems.

Necessity of the Project:

Understanding the potential distribution in capacitors is vital for:

- Enhancing the design of capacitive devices in power systems, electronics, and sensors.
- Optimizing energy storage solutions.
- Predicting and mitigating electric field-related failures in high-voltage applications.

Traditional analytical methods often fail to capture the complexities of real-world scenarios involving irregular geometries or inhomogeneous materials. This is particularly significant in industries like high-voltage engineering, where the design of insulators and capacitors often involves non-uniform shapes, or in medical imaging, where varying material properties affect electromagnetic field interactions. For instance, determining the potential distribution in capacitors with curved or uneven plates, or those embedded in materials with non-uniform permittivity, often requires numerical methods to achieve accurate solutions. This project leverages FDM to compute the potential distribution between capacitor plates, providing a robust and adaptable approach to solving such problems.

Project Implementation

The project employs the FDM to calculate the potential distribution for a simple parallel-plate capacitor configuration. The computational model solves the Laplace equation:

$$\frac{\partial^2 \psi}{\partial x^2} + \frac{\partial^2 \psi}{\partial y^2} = 0$$

in a two-dimensional domain with appropriate boundary conditions.

Methodology

1. **Discretization of the Domain:** The domain is divided into a grid of nodes, and the Laplace equation is approximated using central differences.
2. **Boundary Conditions:** Dirichlet boundary conditions are applied to simulate the potential on the plates, while other boundaries are assumed to be at zero potential.
3. **Iterative Solver:** The system of equations is solved iteratively using techniques such as Gauss-Seidel or Successive Over-Relaxation (SOR).
4. **Visualization:** The computed potential values are visualized to analyze the field distribution.

Successive Over-Relaxation (SOR)

Method

The **Successive Over-Relaxation (SOR)** method is an enhancement of the Gauss-Seidel iterative technique for solving systems of linear equations ($Ax=b$), commonly arising in numerical solutions of PDEs. It introduces a relaxation factor (ω) to speed up convergence:

$$x_i^{(k+1)} = (1 - \omega)x_i^{(k)} + \frac{\omega}{a_{ii}}(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)})$$

Key Points:

- **Relaxation Factor (ω):**
 - $\omega > 1$: Over-relaxation (faster convergence).
 - $\omega = 1$: Reduces to Gauss-Seidel.
 - $\omega < 1$: Under-relaxation (used for stability).
- **Convergence:** Achieved if $0 < \omega < 2$, and the matrix is diagonally dominant or symmetric positive definite.
- **Stopping Criterion:** Iterations stop when

$$\|x^{(k+1)} - x^{(k)}\| < \epsilon$$

where ϵ is the tolerance.

Advantages:

- Faster convergence compared to Gauss-Seidel.
- Effective for large systems, particularly in sparse matrices from PDE discretizations.

SOR can be implemented as:

```
def SOR(A, b, x0, omega, tol, max_iter):  
    x = x0.copy()  
    for _ in range(max_iter):  
        x_old = x.copy()  
        for i in range(len(b)):  
            sigma = sum(A[i, j] * x[j] for j in range(len(b)) if j != i)  
            x[i] = (1 - omega) * x[i] + omega * (b[i] - sigma) / A[i, i]  
        if np.linalg.norm(x - x_old, np.inf) < tol:  
            break  
    return x
```

Code Implementation

The code computes the potential distribution in a parallel-plate capacitor using the Finite Difference Method, demonstrating the effectiveness of this numerical approach. The complete code implementation is available on GitHub: [GitHub Repository: FDM-based Node Potential Calculation for Capacitor Plates](#)

Results

The computed potential distribution demonstrates a uniform gradient between the plates, as expected for a parallel-plate capacitor, validating the numerical approach and its alignment with theoretical predictions. The visualization confirms the validity of the FDM approach in solving electrostatic field problems.

Code Snippet

Using the Successive Over-Relaxation (SOR) method:

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

def calculate_inner_node_voltages(N, top_voltage, omega=1.5):
    E = top_voltage / (N - 1)
    boundary_top = [top_voltage] * N
    boundary_bottom = [0] * N
    boundary_left = [top_voltage - i * E for i in range(1, N
- 1)]
    boundary_right = [top_voltage - i * E for i in range(1, N
- 1)]

    voltages = np.zeros((N, N))
    voltages[0, :], voltages[-1, :], voltages[1:-1, 0],
    voltages[1:-1, -1] = boundary_top, boundary_bottom,
    boundary_left, boundary_right

    max_iterations, tolerance = 10000, 1e-6
    for iteration in range(max_iterations):
        old_voltages = np.copy(voltages)
        for i in range(1, N-1):
            for j in range(1, N-1):
                voltages[i, j] = (1 - omega) * old_voltages[i,
j] + omega * 0.25 * (voltages[i+1, j] + voltages[i-1, j] +
    voltages[i, j+1] + voltages[i, j-1])
            if np.linalg.norm(voltages - old_voltages) < tolerance:
                break
    return voltages

top_voltage, N, omega = 500, 6, 1.5
voltages = calculate_inner_node_voltages(N, top_voltage,
    omega)

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
x, y = np.linspace(0, N-1, N), np.linspace(0, N-1, N)
X, Y = np.meshgrid(x, y)
```

```

surf = ax.plot_surface(X, Y, voltages, cmap='viridis',
edgecolor='k')
ax.plot_surface(X, Y, top_voltage * np.ones_like(voltages),
color='red', alpha=0.6)
ax.plot_surface(X, Y, np.zeros_like(voltages), color='blue',
alpha=0.6)

ax.set_title(f"Voltage Distribution (Top: {top_voltage}V,
Bottom: 0V)")
ax.set_xlabel("X Position")
ax.set_ylabel("Y Position")
ax.set_zlabel("Voltage (V)")

fig.colorbar(surf, ax=ax, shrink=0.5, aspect=5)
plt.show()

```

Provided Input

```

Enter the top boundary voltage (e.g., 500V): 600
Enter the number of grid points per side, including boundary (e.g., 5): 6
Enter the relaxation factor (e.g., 1.5): 1.6
Converged after 42 iterations with relaxation factor 1.6.

```

For using the **SOR factor (1.6)** here this problem converges after **42 iterations**

Output

```

Final node voltages:
[[600.         600.         600.         600.         600.
  600.         600.        ]
 [500.         500.00000017 500.00000006 500.00000001 500.00000009
 500.00000004 500.        ]
 [400.         400.00000002 399.99999989 399.99999993 400.
 400.00000003 400.        ]
 [300.         299.99999986 299.99999993 299.99999988 300.00000005
 300.00000004 300.        ]
 [200.         200.00000001 199.9999999 200.         200.00000009
 200.00000004 200.        ]
 [100.         99.99999984 99.99999996 100.00000001 100.00000006
 100.         100.        ]
 [  0.          0.          0.          0.          0.
  0.          0.        ]]

```

Without Using the Successive Over-Relaxation (SOR) method:

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

def calculate_inner_node_voltages(N, top_voltage):
    E = top_voltage / (N - 1)
    boundary_top = [top_voltage] * N
    boundary_bottom = [0] * N
    boundary_left = [top_voltage - i * E for i in range(1, N
- 1)]
    boundary_right = [top_voltage - i * E for i in range(1, N
- 1)]

    voltages = np.zeros((N, N))
    voltages[0, :], voltages[-1, :], voltages[1:-1, 0],
    voltages[1:-1, -1] = boundary_top, boundary_bottom,
    boundary_left, boundary_right

    for _ in range(10000):
        old_voltages = np.copy(voltages)
        for i in range(1, N-1):
            for j in range(1, N-1):
                voltages[i, j] = 0.25 * (voltages[i+1, j] +
    voltages[i-1, j] + voltages[i, j+1] + voltages[i, j-1])
                if np.linalg.norm(voltages - old_voltages) < 1e-6:
                    break
    return voltages

N, top_voltage = 6, 500
voltages = calculate_inner_node_voltages(N, top_voltage)

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
x, y = np.linspace(0, N-1, N), np.linspace(0, N-1, N)
X, Y = np.meshgrid(x, y)

surf = ax.plot_surface(X, Y, voltages, cmap='viridis',
    edgecolor='k')
ax.plot_surface(X, Y, top_voltage * np.ones_like(voltages),
    color='red', alpha=0.6)
```

```

ax.plot_surface(X, Y, np.zeros_like(voltages), color='blue',
alpha=0.6)

ax.set_title(f"Voltage Distribution ({int(top_voltage)}V Top,
Grounded Bottom)")
ax.set_xlabel("X Position")
ax.set_ylabel("Y Position")
ax.set_zlabel("Voltage (V)")
fig.colorbar(surf, ax=ax, shrink=0.5, aspect=5)
plt.show()

```

Provided Input

```

Enter the top boundary voltage (e.g., 500V): 600
Enter the number of grid points per side, including boundary (e.g., 5): 6
Converged after 70 iterations.

```

When we don't use the **SOR factor** here this problem converges after **70 iterations**

Output

```

Final node voltages:
[[600.      600.      600.      600.      600.
  600.      600.      ]
 [500.      499.9999965 499.9999948 499.9999948 499.9999961
  499.9999981 500.      ]
 [400.      399.9999948 399.9999922 399.9999922 399.9999942
  399.9999971 400.      ]
 [300.      299.9999948 299.9999922 299.9999922 299.9999942
  299.9999971 300.      ]
 [200.      199.9999961 199.9999942 199.9999942 199.9999956
  199.9999978 200.      ]
 [100.      99.9999981  99.9999971  99.9999971  99.9999978
  99.9999989 100.      ]
 [  0.      0.      0.      0.      0.
   0.      0.      ]]

```

Conclusion from the Outputs

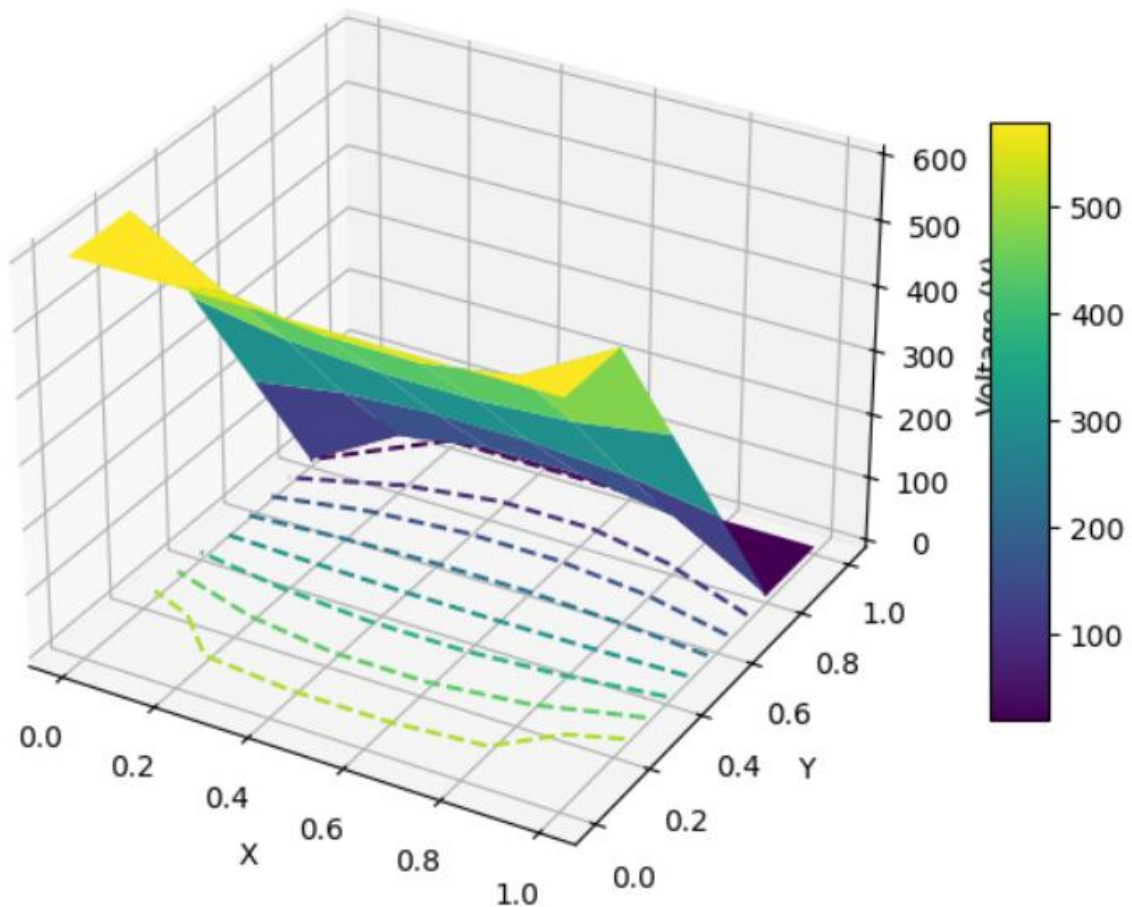
By incorporating the Successive Over-Relaxation (SOR) method, the number of iterations required to compute the potential distribution at the nodes significantly decreases. This improvement arises because the SOR method modifies the iterative update rule by introducing a relaxation factor (ω), which helps accelerate convergence towards the solution. For the same problem, the SOR method demonstrates better computational efficiency, as it effectively reduces redundant updates in each iteration, making it a preferred choice for solving large-scale grid-based potential problems.

Voltage Distribution and Error Analysis Between Capacitor Plates

We are using **3D surface plots** to visualize:

1. **Numerical Voltage Distribution:** Shows how the voltage varies across the grid.
2. **Error Distribution:** Displays the absolute error between the numerical and analytical solutions.

Numerical Voltage Distribution

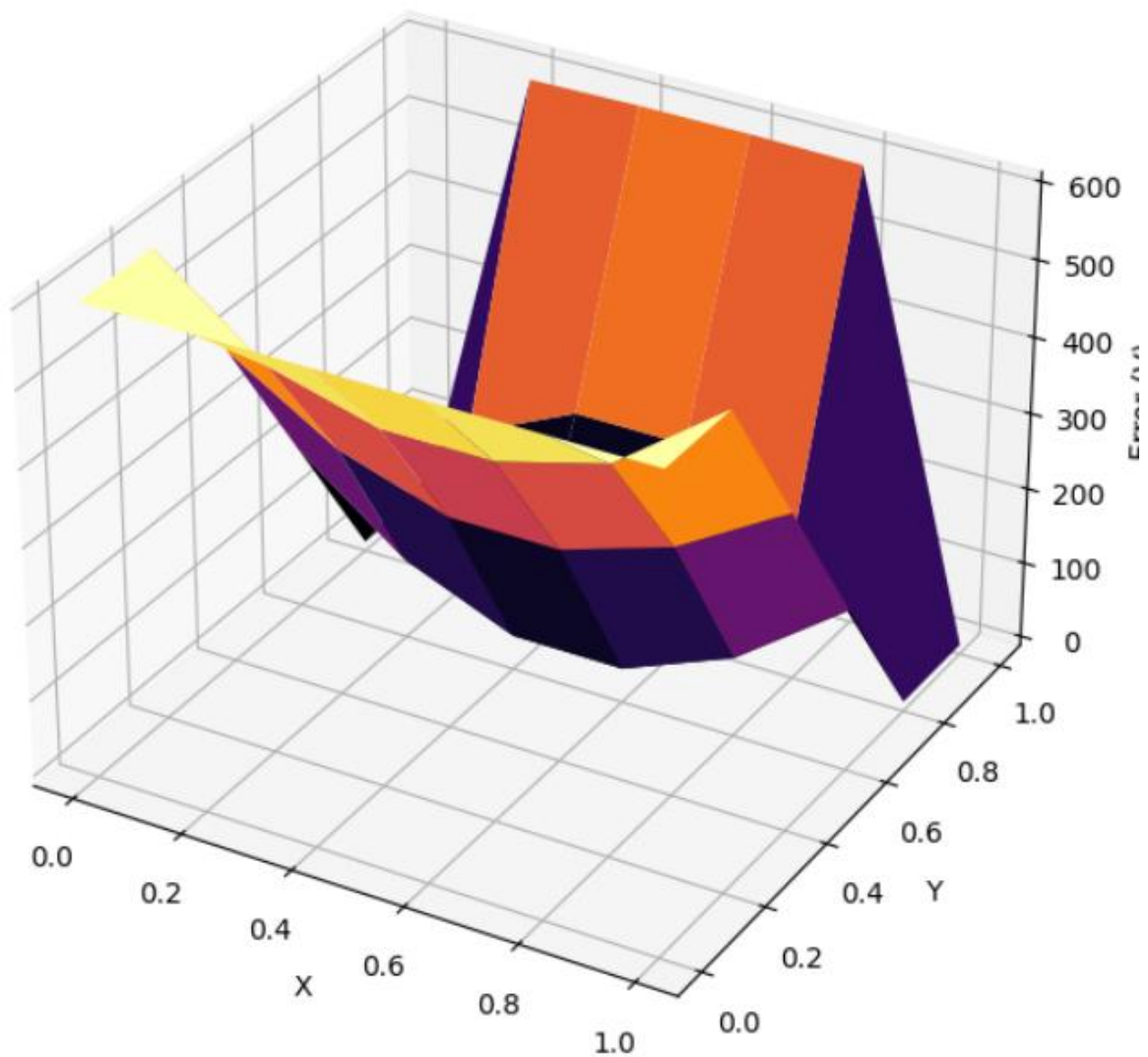


Provided Input

Enter the top plate voltage (e.g., 100): 600

Enter the number of grid points per side (e.g., 20): 6

Error Distribution



2D and 3D Visualization of Potential Distribution Between Capacitor Plates

Using Neumann Boundary Conditions:

Neumann boundary conditions can also be applied in the Finite Difference Method (FDM) by specifying the normal derivative of the potential at a boundary instead of the potential itself. For example, at a boundary, the potential can be updated using the finite difference approximation of the gradient, Such as:

$$V(0, y, z) = V(1, y, z) - \Delta z \left(\frac{\partial V}{\partial z} \right)$$

This modifies the potential values at the boundary based on the known gradient, allowing for accurate modeling of the system with derivative-based boundary conditions.

Where:

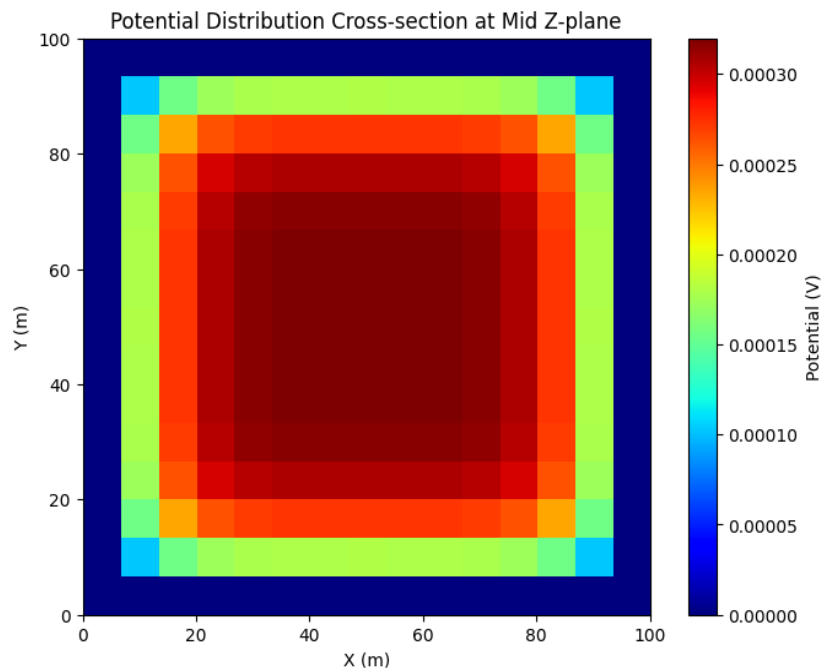
- $V(0, y, z)$ is the potential at the boundary node.
- Δz is the grid spacing in the z-direction.
- $\left(\frac{\partial V}{\partial z} \right)$ is the specified normal derivative (the gradient).

Neumann boundary condition can be implemented as:

To implement the **Neumann boundary condition** in your FDM code, you can modify the potential at the boundary by using the gradient approximation. For example, at $Z=0$, the potential is updated as:

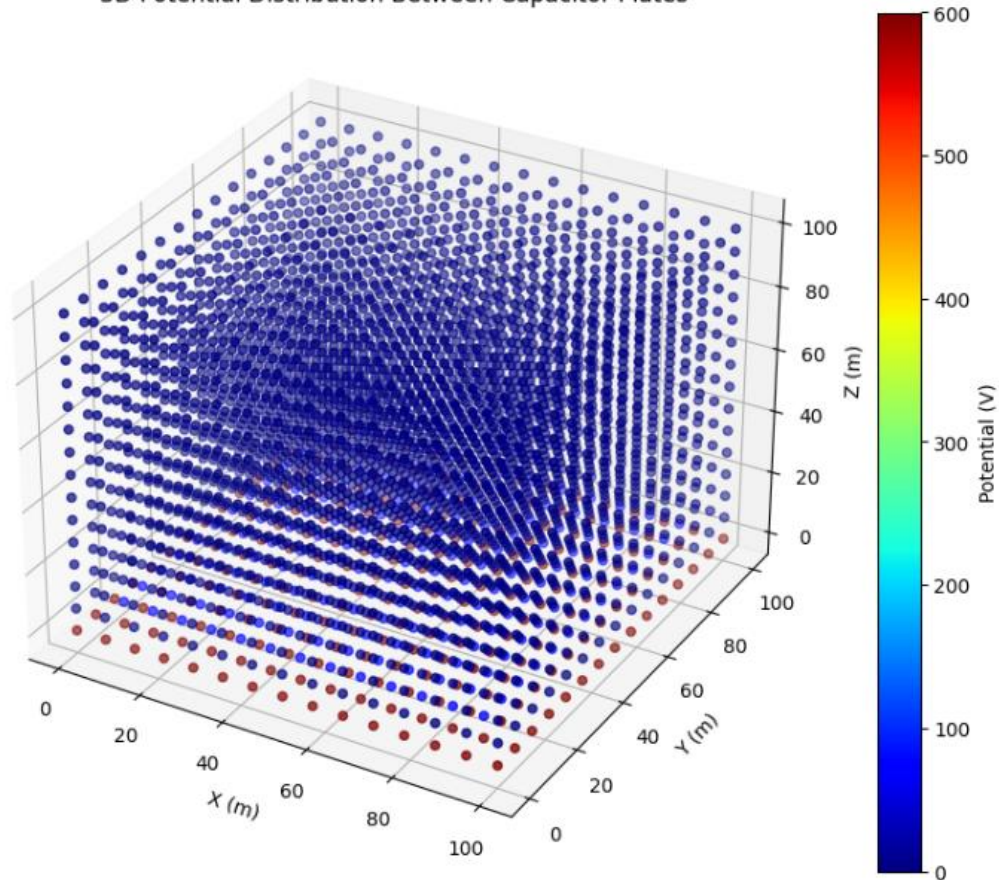
```
# Apply Neumann condition at Z=0 (normal derivative known)
V[:, :, 0] = V[:, :, 1] - dz * normal_derivative_value
```

Here, **normal_derivative_value** is the known normal derivative (gradient) at the boundary, and dz is the grid spacing in the z -direction. This updates the potential at the boundary based on the gradient.



2D Visualization

3D Potential Distribution Between Capacitor Plates



Colors represent electric potential:

- **Red:** High potential (near the bottom plate).
- **Blue:** Low potential (near the top plate).
- **Intermediate Colors:** Gradual transition between the plates.

Points: Represent nodes where potential is calculated using FDM.

Physical Meaning: Show potential decrease and electric field strength (rapid color change = stronger field).

Uniformity: Near-linear gradient indicates a uniform electric field between plates.

Future Developments:

- **Integration with Complex Geometries:** Extending the model to handle irregular capacitor shapes.
- **Material Inhomogeneity:** Incorporating dielectric materials with varying permittivity.
- **Three-Dimensional Analysis:** Expanding the solution to 3D domains for more comprehensive studies.
- **Optimization Algorithms:** Employing machine learning techniques to optimize capacitor design.

Conclusion

This project highlights the effectiveness of the Finite Difference Method in computing electrostatic fields. The results provide valuable insights into capacitor design and pave the way for advanced computational studies in electromagnetics.